

- [Markdown Document Production Line SOP](#)
 - [1. Purpose & Success Criteria](#)
 - [Success criteria](#) 
 - [2. Scope & Non-goals](#)
 - [In scope](#)
 - [Out of scope](#)
 - [3. Project Structure](#)
 - [Folder tree](#)
 - [Key files overview](#)
 - [4. Process Overview](#)
 - [4.1 High-level flowchart \(Mermaid\)](#)
 - [4.2 Roles & responsibilities](#)
 - [5. Authoring Standards \(Markdown Style Guide\)](#)
 - [5.1 Headings](#)
 - [5.2 Emphasis & inline code](#)
 - [5.3 Lists \(nested\)](#)
 - [5.4 Links & images](#)
 - [6. Step-by-step Workflow](#)
 - [Step 1 - Write content](#)
 - [Step 2 - Run quality checks \(lightweight\)](#)
 - [Step 3 - Render HTML](#)
 - [Step 4 - Render PDF](#)
 - [Step 5 - Publish \(GitHub Classroom\)](#)
 - [Step 6 - Quality Gate \(Definition of Done\)](#)
 - [7. Troubleshooting](#)
 - [Reference build commands](#)
 - [8. Math & Notes](#)
 - [9. Appendix: Example "Nice-looking" Elements](#)
 - [9.1 Small callout](#)
 - [9.2 Mini table](#)
 - [9.3 Tiny snippet](#)

Markdown Document Production Line SOP

A reproducible pipeline for writing → validating → rendering → publishing

Owner: Hsin-En, Tsai

Course: Generative AI Application Systems and Engineering (AIASE 2026) **Last updated:** 2026-03-04

Outputs: PDF / HTML

Toolchain: Markdown + Pandoc + CSS + Mermaid CLI + Playwright (Chromium)

1. Purpose & Success Criteria

This SOP defines a **document production line** that converts a single Markdown source into professional outputs (PDF/HTML) with **repeatable commands** and **quality checks**.

Success criteria

- ✓ A single source of truth: `content.md`
- ✓ Reproducible rendering commands (fresh environment friendly)
- ✓ At least **two outputs** in `output/` (e.g., `output.pdf` and `output.html`)
- ✓ Content demonstrates rich Markdown features (headings, tables, code blocks, diagrams, math, etc.)
- ✓ Clear troubleshooting section for common failures

Design principle: *Decouple content from rendering.*

Write once, render many times.

2. Scope & Non-goals

In scope

- Markdown authoring conventions
- Project folder structure
- Rendering to HTML and PDF
- Quality checks and troubleshooting

Out of scope

- Advanced CI/CD deployment (optional)
- Multi-author merge workflow (optional)
- Website hosting (optional)

3. Project Structure

Folder tree

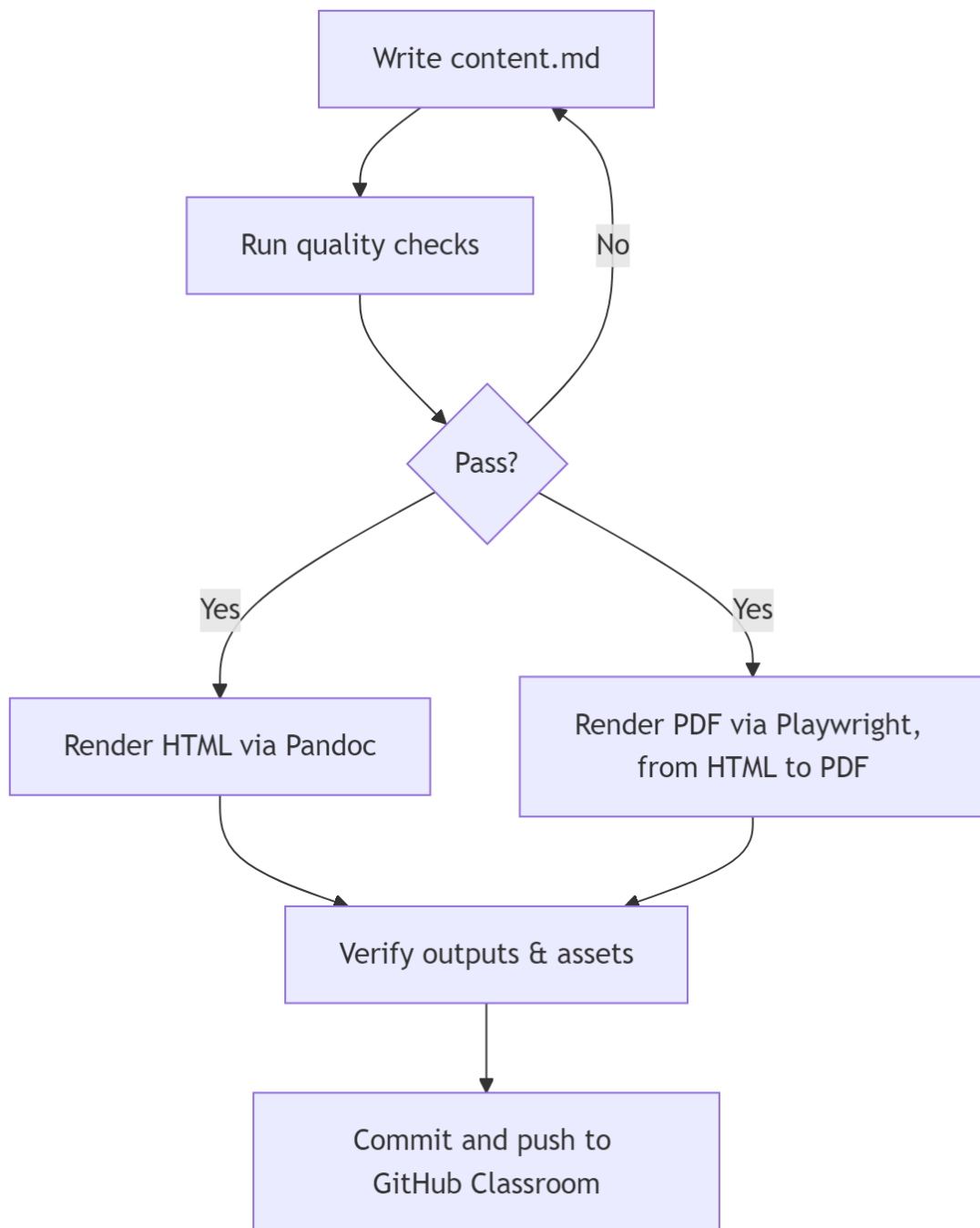
```
AIASE2026-HW1/  
├─ content.md  
├─ README.md  
├─ requirements.txt      (optional)  
└─ output/  
    ├─ output.html  
    └─ output.pdf
```

Key files overview

File/Folder	Purpose	Must?
content.md	Main document source (single source of truth)	✓ Yes
README.md	Reproduction guide (install + commands + notes)	✓ Yes
requirements.txt	Python deps (only if you use Python steps)	● Optional
output/	Rendered artifacts	✓ Yes

4. Process Overview

4.1 High-level flowchart (Mermaid)



Workflow flowchart

4.2 Roles & responsibilities

- **Author:** writes/updates `content.md`
- **Renderer:** executes Pandoc commands and verifies outputs
- **Reviewer (self-check):** confirms reproducibility and structure

5. Authoring Standards (Markdown Style Guide)

5.1 Headings

Using consistent hierarchy: # for title → ## for sections → ### for subsections.

5.2 Emphasis & inline code

- Use **bold** for keywords
- Use *italic* for notes
- Use ~~striketrough~~ for deprecated items
- Use `inline code` for commands, filenames, flags

5.3 Lists (nested)

1. Draft

- Outline sections
- Add diagrams

2. Validate

- Links
- Code blocks

3. Render

- HTML
- PDF

5.4 Links & images

- Example link: [Pandoc User's Guide](#)
- Example image placeholder:

Markdown Document Production Line SOP

A reproducible pipeline for writing → validating → rendering → publishing

Owner: Hsin-En, Tsai

Course: Generative AI Application Systems and Engineering (AIASE 2026) **Last updated:** 2026-03-04

Outputs: PDF / HTML

Toolchain: Markdown + Pandoc + Mermaid CLI + Playwright (Chromium)

Markdown Document Production Line SOP

1. Purpose & Success Criteria
 - Success criteria 
2. Scope & Non-goals
 - In scope
 - Out of scope
3. Project Structure
 - Folder tree
 - Key files overview
4. Process Overview
 - 4.1 High-level flowchart (Mermaid)
 - 4.2 Roles & responsibilities
5. Authoring Standards (Markdown Style Guide)
 - 5.1 Headings
 - 5.2 Emphasis & inline code
 - 5.3 Lists (nested)
 - 5.4 Links & images
6. Step-by-step Workflow
 - Step 1 - Write content
 - Step 2 - Run quality checks (lightweight)
 - Step 3 - Render HTML
 - Step 4 - Render PDF
 - Step 5 - Publish (GitHub Classroom)
 - Step 6 - Quality Gate (Definition of Done)
7. Troubleshooting
8. Math & Notes
9. Appendix: Example “Nice-looking” Elements
 - 9.1 Small callout
 - 9.3 Tiny snippet

Tip: Put screenshots into output/ and reference them relatively.

6. Step-by-step Workflow

Step 1 - Write content

Edit `content.md` with a Markdown editor (VS Code recommended).

Checklist - At least 6 top-level sections - At least 2 tables - At least 2 code blocks (with language tags) - Mermaid diagram present - Task list present

Step 2 - Run quality checks (lightweight)

2.1 Manual checks

- Headings are properly nested
- Tables render correctly
- Links are valid
- Workflow diagram image (`flowchart.png`) is present (generated from Mermaid source).

2.2 Automated link check (Python)

It's an extra engineering touch.

```
# tools/check_outputs.py
from pathlib import Path

required = [
    Path("content.md"),
    Path("output/output.html"),
    Path("output/output.pdf"),
]

missing = [p for p in required if not p.exists()]
if missing:
    raise SystemExit(f"Missing files: {missing}")
print("OK: required files exist.")
```

Step 3 - Render HTML

```
mkdir -p output
pandoc content.md -s --toc -o output/output.html
```

Styling: assets/style.css is applied via Pandoc -c to improve readability of headings, tables, code blocks, and blockquotes.

Verification

- ☐ output/output.html exists
- ☐ Open it in a browser (check spacing, lists, tables)

Step 4 - Render PDF

```
node tools/print_pdf.js
```

PDF is generated from output/output.html using headless Chromium (Playwright) for better reproducibility (no LaTeX required).

Verification

- ☐ output/output.pdf exists
- ☐ Table layout is readable
- ☐ Code blocks are not cut off badly

Step 5 - Publish (GitHub Classroom)

```
git add content.md README.md output/
git commit -m "HW1: reproducible markdown production line"
git push
```

Step 6 - Quality Gate (Definition of Done)

6.1 Output checklist

- ☒ HTML output generated
- ☒ PDF output generated
- ☒ README includes exact commands
- ☒ Folder structure matches requirement
- ☒ Rendered results placed in output/

6.2 Content richness checklist (for grading)

- ☒ Multi-level headings
- ☒ Emphasis (bold/italic/strike)
- ☒ Lists (ordered/unordered/nested)
- ☒ Tables
- ☒ Code blocks with language tags
- ☒ Blockquotes
- ☒ Horizontal rule
- ☒ Links and images
- ☒ Task list
- ☒ Math formula (below)
- ☒ Mermaid diagram

7. Troubleshooting

Symptom	Likely cause	Fix
pandoc is not recognized	Pandoc not installed / PATH not refreshed	Install Pandoc, then reopen terminal and run <code>pandoc --version</code>
Mermaid image not showing in HTML/PDF	Image path is wrong (HTML is in output/) or image not committed	Put images in output/ and reference as <code>![...] (flowchart.png)</code> (NOT <code>output/flowchart.png</code>); commit the image files

Symptom	Likely cause	Fix
mmdc is not recognized	Mermaid CLI not installed	Install: <code>npm install -g @mermaid-js/mermaid-cli</code> , then reopen terminal
Mermaid parse error	Node labels contain special characters (e.g., <code>-></code> , parentheses)	Simplify labels (e.g., use HTML to PDF and avoid parentheses)
PDF build fails with Cannot find module 'playwright'	Playwright not installed in project	Run: <code>npm init -y</code> then <code>npm install playwright</code>
PDF build fails with browser missing	Chromium not installed for Playwright	Run: <code>npx playwright install chromium</code>
PDF output looks different from VS Code preview	Different renderers/styles	Use the generated output/output.html as the source for PDF (Playwright keeps HTML/PDF consistent)

Reference build commands

```
# 1) Mermaid (.mmd) -> PNG
mmdc -i assets/flowchart.mmd -o output/flowchart.png -b transparent
      -s 2

# 2) Markdown -> HTML
pandoc content.md -s --toc -o output/output.html

# 3) HTML -> PDF (no LaTeX required)
node tools/print_pdf.js
```

8. Math & Notes

A simple cost model for rendering iterations:

$$T_{total} = N \cdot (T_{edit} + T_{render} + T_{verify})$$

Where: - N = number of iterations - T_{edit} = editing time - T_{render} = rendering time - T_{verify} = verification time

9. Appendix: Example “Nice-looking” Elements

9.1 Small callout

✓ Best practice: keep `content.md` content-focused and move instructions to `README.md`.

9.2 Mini table

Item	Value
Renderer	Pandoc
Outputs	HTML, PDF
Reproducibility	One-command build

9.3 Tiny snippet

```
pandoc content.md -s --toc -c ../assets/style.css -o
    output/output.html
node tools/print_pdf.js
```